User Manual

for S32 PCIE Driver

Document Number: UM11PCIEASR4.4 Rev0000R4.0.2 Rev. 1.0

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

This User Manual describes NXP Semiconductor Pcie for S32. Pcie driver configuration parameters and deviations from the specification are described in Driver chapter of this document. Pcie driver requirements and APIs are described in the Pcie driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSW	Basic Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
ECU	Electronic Control Unit
ISR	Interrupt Service Routine
N/A	Not Applicable
VLE	Variable Length Encoding

2.5 Reference List

#	Title	Version
2	S32G Reference Manual	S32G2 Reference Manual, Rev. 7, February 2023
		S32G3 Reference Manual, Rev.3, Feb 2023
3	Datasheet	S32G2 Data Sheet, Rev 6, Dec 2022
		S32G3 Data Sheet, Rev 2, RC, Feb 2023
		VR5510 Data Sheet, Rev 5, April 2022
3	Errata	S32G2: Mask Set Errata for Mask 0P77B v3.0
		S32G2: Mask Set Errata for Mask 1P77B v1.0
		S32G3: Mask Set Errata for Mask 1P72B v2.1

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See Table Reference List).

3.2 Driver Design Summary

The Pcie driver is implemented as an Autosar complex device driver. It uses the Pcie module present on S32G family MCUs and the Serdes hardware peripheral which acts as PHY for SGMII and PCIe. Hardware and software settings can be configured using an Autosar standard configuration tool.

PCIE functionalities:

- initialization in endpoint mode (Pcie_Init), including configuration of BAR sizes and attributes, and configuration of inbound windows in the internal address translation unit (IATU) for mapping of BAR address spaces on user-provided memory regions.
- configuration of outbound regions at runtime (Pcie_SetOutboundRegion), for accessing shared memory in the root complex
- asynchronous DMA read/write operations (Pcie_DmaRead, Pcie_DmaCheckReadStatus, Pcie_DmaWrite, Pcie_DmaCheckWriteStatus) targetting shared memory in the root complex
- generation of MSI/MSIx interrupts
- optionally interrupts can be activated for generating notifications for DMA transfer complete events

3.3 Hardware Resources

The hardware configured by the Pcie driver is the same between derivatives.

Pcie Modules:

```
S32G2XX: Pcie_0, Pcie_1
S32G3XX: Pcie_0, Pcie_1
```

3.4 Deviations from Requirements

N/A

3.5 Driver Limitations

- when configuring PCIe, do not use BARs 2 and 3, as they have special functions. Use BARs 0, 1, 5, 6
- BAR 1, if used, has a fixed size of 64KB
- Multicore support is not implemented.
- Do not mix interrupts and polling for handling DMA events on the same PCIe controller
- When a read/write DMA channel is used in linked list mode and a new transfer request is added into an almost empty list, there is a possible race condition which can cause this transfer to stall until anoher transfer request is made. The linked list mode is intended for transfers which are performed often and are not time-sensitive. For rare/time sensitive transfers use DMA in normal mode.
- [ERR051579] ensure that the remote partner does not generate request TLP with TPH[1] bit set to 1b when accessing DMA configuration registers
- [ERR051592] if the LTSSM recovery is triggered during L2 negotiation, the controller enters ASPM L1 regardless of ASPM being enabled or not for the port. Consequently, the controller takes a longer time to exit L2
- [ERR051585] in order to avoid getting the controller stuck in loopback mode, follow **one** of the following workarounds:
  - avoid lane reversal in loopback mode
  - avoid speed transition (from Gen1/Gen2 to Gen3 speed)
  - ensure that the remote Loopback Master sends TS OSs with the Loopback bit and the Compliance Receive bit set to 1b after LTSSM kicking off.
- [ERR051594] if the controller enters in RASDP Error Mode and a MSIX request is triggered, then it blocks the consequent register accesses; any attempt to access a register via APB interface will lead to timeout; the timeout can be observed by reading PE0_ERR_STS [APBSLV_TIMEOUT_STS] which will be set to 1b.

3.6 Driver usage and configuration tips

The s32 SoCs have two PCIe controllers. Each of them has a corresponding SerDes module which functions as PHY for PCIe. In order to use PCIe you will also need to use the SerDes driver to configure and initialize the correspondind SerDes instance.

3.7 Runtime errors

The driver generates the following DET errors at runtime.

Table 3.1 Default Errors (reported by DET)

Function	Error Code	Condition triggering the error
Pcie_Init	PCIE_E_INIT_DONE	Driver already initialized.
Pcie_SetOutboundRegion Pcie_DmaRead Pcie_DmaReadIntEnable Pcie_DmaCheckReadStatus Pcie_DmaWrite Pcie_DmaWriteIntEnable Pcie_DmaCheckWriteStatus Pcie_SendMsi Pcie_SendMsiX Pcie_DmaLlReadSetup Pcie_DmaLlWriteSetup	PCIE_E_UNINIT	Driver not yet initialized.
Pcie_Init Pcie_SetOutboundRegion Pcie_DmaRead Pcie_DmaRead Pcie_DmaReadIntEnable Pcie_DmaReadIntEnable Pcie_DmaReadIntEnable Pcie_DmaReadIntEnable Pcie_DmaCheckReadStatus Pcie_DmaWrite Pcie_DmaWriteIntEnable Pcie_DmaCheckWriteStatus Pcie_SendMsi Pcie_SendMsiX Pcie_DmaLlReadSetup Pcie_DmaLlWriteSetup Pcie_GetVersionInfo	PCIE_E_INVALID_PARAM	API service called with invalid parameter.

Function	Error Code	Condition triggering the error
Pcie_Init Pcie_SetOutboundRegion Pcie_DmaRead Pcie_DmaRead Pcie_DmaReadIntEnable Pcie_DmaReadIntEnable Pcie_DmaReadIntEnable Pcie_DmaReadIntEnable Pcie_DmaCheckReadStatus Pcie_DmaWrite Pcie_DmaWriteIntEnable Pcie_DmaCheckWriteStatus Pcie_SendMsi Pcie_SendMsiX Pcie_DmaLlReadSetup Pcie_DmaLlWriteSetup	PCIE_E_PARAM_CONFIG	Invalid configuration.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

#define <Mip>Conf_<Container_ShortName>_<Container_ID>

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module Pcie
  - Container PcieGeneral
    - Parameter PcieDevErrorDetect
    - Parameter PcieEnableUserModeSupport
    - Parameter PcieMulticoreSupport
  - Container PcieGlobalConfig
    - Reference PcieEcucPartitionRef
    - Container APIconfiguration
      - Parameter PcieSetOutboundRegionApi
      - Parameter PcieDmaReadApi
      - Parameter PcieDmaReadIntEnableApi
      - Parameter PcieDmaCheckReadStatusApi
      - Parameter PcieDmaWriteApi
      - Parameter PcieDmaWriteIntEnableApi
      - Parameter PcieDmaCheckWriteStatusApi
      - Parameter PcieSendMsiApi
      - Parameter PcieSendMsiXApi
      - Parameter PcieDmaLlReadSetupApi
      - Parameter PcieDmaLlWriteSetupApi
      - Parameter PcieVersionInfoApi
    - Container PcieChannel
      - Parameter PcieChannelId
      - Parameter PcieHwChannel
      - Parameter PcieClass
      - Parameter MsiCount
      - Parameter MsixEnabled
      - Parameter DmaReadDoneHandler
      - Parameter DmaReadErrorHandler
      - Parameter DmaWriteDoneHandler
      - Parameter DmaWriteErrorHandler

        - Reference PcieChannelEcucPartitionRef
        - Container BAR0Configuration
        - Parameter Bar0Type64Bit
        - Parameter Bar0TypeIo
        - Parameter Bar0TypePrefetchable
        - Parameter Bar0Size
        - Parameter Bar0StartAddr
        - Parameter Bar0StartSymbol
        - Container BAR1Configuration
        - Parameter Bar1TypeIo
        - Parameter Bar1Size
        - Parameter Bar1StartAddr
        - Parameter Bar1StartSymbol
        - Container BAR4Configuration
        - Parameter Bar4Type64Bit
        - Parameter Bar4TypeIo
        - Parameter Bar4TypePrefetchable
        - Parameter Bar4Size
        - Parameter Bar4StartAddr
        - Parameter Bar4StartSymbol
        - Container BAR5Configuration
        - Parameter Bar5TypeIo
        - Parameter Bar5Size
        - Parameter Bar5StartAddr
        - Parameter Bar5StartSymbol
    - Container CommonPublishedInformation
        * Parameter ArReleaseMajorVersion
        * Parameter ArReleaseMinorVersion
        * Parameter ArReleaseRevisionVersion
        * Parameter ModuleId
        * Parameter SwMajorVersion
        * Parameter SwMinorVersion
        * Parameter SwPatchVersion
        * Parameter VendorApiInfix
        * Parameter VendorId

4.1 Module Pcie

Vendor specific: Configuration of the Pcie (PcieDriver) module

Included containers:

- PcieGeneral
- PcieGlobalConfig
- CommonPublishedInformation

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-PRE-COMPILE

4.2 Container PcieGeneral

Container for common configuration options

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter PcieDevErrorDetect

Vendor specific: Switches the development error detection and notification on or off.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.4 Parameter PcieEnableUserModeSupport

Vendor specific: User Mode not supported.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.5 Parameter PcieMulticoreSupport

Vendor specific: This parameter globally enables the possibility to support multicore.

If PcieMulticoreSupport is disabled, then for all the variants no EcucPartition shall be defined.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.6 Container PcieGlobalConfig

Vendor specific: This container contains the global configuration parameter of the Pcie driver. This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exit once per configuration set.

Included subcontainers:

- APIconfiguration
- PcieChannel

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.7 Reference PcieEcucPartitionRef

Vendor specific: Maps the Pcie driver to zero or multiple ECUC partitions to make the

modules API available in this partition. The Pcie driver will operate as

an independent instance in each of the partitions.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.8 Container APIconfiguration

Vendor specific: This container contains the configuration of the available APIs

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.9 Parameter PcieSetOutboundRegionApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_SetOutboundRegion function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.10 Parameter PcieDmaReadApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_DmaRead function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.11 Parameter PcieDmaReadIntEnableApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_DmaReadIntEnable function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.12 Parameter PcieDmaCheckReadStatusApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_DmaCheckReadStatus function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.13 Parameter PcieDmaWriteApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_DmaWrite function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.14 Parameter PcieDmaWriteIntEnableApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_DmaWriteIntEnable function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.15 Parameter PcieDmaCheckWriteStatusApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_DmaCheckWriteStatus function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.16 Parameter PcieSendMsiApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_SendMsi function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.17 Parameter PcieSendMsiXApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_SendMsiX function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.18 Parameter PcieDmaLlReadSetupApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_DmaLlReadSetup function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.19 Parameter PcieDmaLlWriteSetupApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_DmaLlWriteSetup function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.20 Parameter PcieVersionInfoApi

Vendor specific: Pre-processor switch to enable and disable the Pcie_GetVersionInfo function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.21 Container PcieChannel

Vendor specific: This container contains the configuration (parameters) of the Pcie Controller(s).

Note:"User should use unique names for naming the Pcie channels across different PcieGlobalConfig Sets."

Included subcontainers:

- BAR0Configuration
- BAR1Configuration
- BAR4Configuration
- BAR5Configuration

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

4.22 Parameter PcieChannelId

Vendor specific: Identifies the Pcie channel.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1
min	0

4.23 Parameter PcieHwChannel

Vendor specific: Selects the physical Pcie Channel.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Pcie_0
literals	['Pcie_0', 'Pcie_1']

4.24 Parameter PcieClass

Vendor specific: PCIE class to configure in the Class Code register of the PCIe header.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	192937984
max	4294967295
min	0

4.25 Parameter MsiCount

Vendor specific: Number of MSIs supported by the driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	32
min	0

4.26 Parameter MsixEnabled

Vendor specific: Enable MSI-X interrupts.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.27 Parameter DmaReadDoneHandler

Vendor specific: Handler for DMA trasfer done event.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.28 Parameter DmaReadErrorHandler

Vendor specific: Handler for DMA trasfer error event.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.29 Parameter DmaWriteDoneHandler

Vendor specific: Handler for DMA trasfer done event.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.30 Parameter DmaWriteErrorHandler

Vendor specific: Handler for DMA trasfer error event.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.31 Reference PcieChannelEcucPartitionRef

Vendor specific: Maps one single Pcie channel to zero or one ECUC partitions.

The ECUC partition referenced is a subset of the ECUC partitions

where the Pcie driver is mapped to.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.32 Container BAR0Configuration

Vendor specific: This container contains the configuration Of the BAR 0 register.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.33 Parameter Bar0Type64Bit

Vendor specific: Specifies that BAR0 size is 64-bit. This will automatically disable BAR1.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.34 Parameter Bar0TypeIo

Vendor specific: Specifies that BAR0 is an I/O BAR.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.35 Parameter Bar0TypePrefetchable

Vendor specific: Specifies that BAR0 is a prefetchable memory region.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.36 Parameter Bar0Size

Vendor specific: Size of address space requested through BAR0. Set to 0 to disable BAR.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.37 Parameter Bar0StartAddr

Vendor specific: Start adress where address space requested through BAR0 will be mapped.

Note: Only one of "BAR 0 Start Address" and "BAR 0 Start Symbol" may be filled

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.38 Parameter Bar0StartSymbol

Vendor specific: C Symbol referencing the Start adress where address space requested through BAR0 will be mapped.

Note: Only one of "BAR 0 Start Address" and "BAR 0 Start Symbol" may be filled

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	

4.39 Container BAR1Configuration

Vendor specific: This container contains the configuration Of the BAR 1 register.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

4.40 Parameter Bar1TypeIo

Vendor specific: Specifies that BAR1 is an I/O BAR.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.41 Parameter Bar1Size

Vendor specific: Size of address space requested through BAR1. Fixed size of 0x10000.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	65536
max	65536
min	0

4.42 Parameter Bar1StartAddr

Vendor specific: Start adress where address space requested through BAR1 will be mapped.

Note: Only one of "BAR 1 Start Address" and "BAR 1 Start Symbol" may be filled

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.43 Parameter Bar1StartSymbol

Vendor specific: C Symbol referencing the Start adress where address space requested through BAR1 will be mapped.

Note: Only one of "BAR 1 Start Address" and "BAR 1 Start Symbol" may be filled

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	

4.44 Container BAR4Configuration

Vendor specific: This container contains the configuration Of the BAR 4 register.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.45 Parameter Bar4Type64Bit

Vendor specific: Specifies that BAR4 size is 64-bit. This will automatically disable BAR5.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.46 Parameter Bar4TypeIo

Vendor specific: Specifies that BAR4 is an I/O BAR.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.47 Parameter Bar4TypePrefetchable

Vendor specific: Specifies that BAR4 is a prefetchable memory region.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.48 Parameter Bar4Size

Vendor specific: Size of address space requested through BAR4. Set to 0 to disable BAR.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.49 Parameter Bar4StartAddr

Vendor specific: Start adress where address space requested through BAR4 will be mapped.

Note: Only one of "BAR 4 Start Address" and "BAR 4 Start Symbol" may be filled

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.50 Parameter Bar4StartSymbol

Vendor specific: C Symbol referencing the Start adress where address space requested through BAR4 will be mapped.

Note: Only one of "BAR 4 Start Address" and "BAR 4 Start Symbol" may be filled

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	

4.51 Container BAR5Configuration

Vendor specific: This container contains the configuration Of the BAR 5 register.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

4.52 Parameter Bar5TypeIo

Vendor specific: Specifies that BAR5 is an I/O BAR.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.53 Parameter Bar5Size

Vendor specific: Size of address space requested through BAR5. Set to 0 to disable BAR.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.54 Parameter Bar5StartAddr

Vendor specific: Start adress where address space requested through BAR5 will be mapped.

Note: Only one of "BAR 5 Start Address" and "BAR 5 Start Symbol" may be filled

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
defaultValue	0
max	4294967295
min	0

4.55 Parameter Bar5StartSymbol

Vendor specific: C Symbol referencing the Start adress where address space requested through BAR5 will be mapped.

Note: Only one of "BAR 5 Start Address" and "BAR 5 Start Symbol" may be filled

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	

4.56 Container CommonPublishedInformation

Common container, aggregated by all modules.

It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.57 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.58 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.59 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.60 Parameter ModuleId

Module ID of this module from Module List.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.61 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.62 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.63 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.64 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the Implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name

Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity >

1. It shall not be used for modules with upper multiplicity =1.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.65 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

This chapter describes the Tresos configuration plug-in for the Pcie Driver. The most of the parameters are described below.

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Chapter 6

Module Documentation

6.1 Pcie HLD

6.1.1 Detailed Description

Macros

- #define PCIE_E_INIT_DONE
  Driver already initialized.
- #define PCIE_E_UNINIT
  Driver not yet initialized
- #define PCIE_E_INVALID_PARAM
  API service called with invalid parameter.
- #define PCIE_E_PARAM_CONFIG
  Invalid configuration.
- #define PCIE_INIT_ID
  Service ID for Pcie_Init() function
- #define PCIE_SETOUTBOUNDREGION_ID
  Service ID for Pcie_SetOutboundRegion() function
- #define PCIE_DMAREAD_ID
  Service ID for Pcie_DmaRead() function
- #define PCIE_DMAREADINTENABLE_ID
  Service ID for Pcie_DmaReadIntEnable() function
- #define PCIE_DMACHECKREADSTATUS_ID
  Service ID for Pcie_DmaCheckReadStatus() function
- #define PCIE_DMAWRITE_ID
  Service ID for Pcie_DmaWrite() function

- #define PCIE_DMAWRITEINTENABLE_ID
  Service ID for Pcie_DmaWriteIntEnable() function
- #define PCIE_DMACHECKWRITESTATUS_ID
  Service ID for Pcie_DmaCheckWriteStatus() function
- #define PCIE_SENDMSI_ID
  Service ID for Pcie_SendMsi() function
- #define PCIE_SENDMSIX_ID
  Service ID for Pcie_SendMsiX() function
- #define PCIE_DMALLREADSETUP_ID
  Service ID for Pcie_DmaLlReadSetup() function
- #define PCIE_DMALLWRITESETUP_ID
  Service ID for Pcie_DmaLlWriteSetup() function
- #define PCIE_GETVERSIONINFO_ID
  Service ID for Pcie_DmaLlWriteSetup() function
- #define PCIE_IATU_REGIONS_ALIGN
  Alignment requirement for IATU inbound/outbound regions

Function Reference

- void Pcie_Init (const Pcie_ConfigType *Config)
  Initializtion of PCIe driver.
- Std_ReturnType Pcie_SetOutboundRegion (uint8 instance, const Pcie_OutRegDescriptorType *outbound↩ Region)
  Creates an outbound address translation region.
- Pcie_StatusType Pcie_DmaRead (uint8 instance, uint8 channel, const Pcie_DmaReadDescriptorType *read↩ Desc)
  Reads data through DMA.
- Std_ReturnType Pcie_DmaReadIntEnable (uint8 instance, uint8 channel, boolean enable)
  Enables DMA read interrupt.
- Pcie_StatusType Pcie_DmaCheckReadStatus (uint8 instance, uint8 channel, uint32 *transfersLeft)
  Checks the status of a DMA read.
- Pcie_StatusType Pcie_DmaWrite (uint8 instance, uint8 channel, const Pcie_DmaWriteDescriptorType *writeDesc)
  Writes data through DMA.
- Std_ReturnType Pcie_DmaWriteIntEnable (uint8 instance, uint8 channel, boolean enable)
  Enables DMA write interrupt..
- Pcie_StatusType Pcie_DmaCheckWriteStatus (uint8 instance, uint8 channel, uint32 *transfersLeft)
  Checks the status of a DMA write.
- Std_ReturnType Pcie_SendMsi (uint8 instance, uint32 intNo)
  Triggers an MSI interrupt.

- Std_ReturnType Pcie_SendMsiX (uint8 instance, uint32 intNo)

  Triggers an MSI-X interrupt.
- Std_ReturnType Pcie_DmaLlReadSetup (uint8 instance, uint8 channel, Pcie_DmaLlElementType *llPtr, uint32 size)

  Prepares a DMA read channel for linked list operation.
- Std_ReturnType Pcie_DmaLlWriteSetup (uint8 instance, uint8 channel, Pcie_DmaLlElementType *llPtr, uint32 size)

  Prepares a DMA write channel for linked list operation.
- void Pcie_GetVersionInfo (Std_VersionInfoType *VersionInfo)

  Returns version information for the driver.

6.1.2 Macro Definition Documentation

6.1.2.1 PCIE_E_INIT_DONE

```
#define PCIE_E_INIT_DONE
```

Driver already initialized.

Definition at line 125 of file Pcie.h.

6.1.2.2 PCIE_E_UNINIT

```
#define PCIE_E_UNINIT
```

Driver not yet initialized

Definition at line 127 of file Pcie.h.

6.1.2.3 PCIE_E_INVALID_PARAM

```
#define PCIE_E_INVALID_PARAM
```

API service called with invalid parameter.

Definition at line 129 of file Pcie.h.

6.1.2.4 PCIE_E_PARAM_CONFIG

```
#define PCIE_E_PARAM_CONFIG
```

Invalid configuration.

Definition at line 131 of file Pcie.h.

6.1.2.5 PCIE_INIT_ID

```
#define PCIE_INIT_ID
```

Service ID for Pcie_Init() function

Definition at line 135 of file Pcie.h.

6.1.2.6 PCIE_SETOUTBOUNDREGION_ID

```
#define PCIE_SETOUTBOUNDREGION_ID
```

Service ID for Pcie_SetOutboundRegion() function

Definition at line 137 of file Pcie.h.

6.1.2.7 PCIE_DMAREAD_ID

```
#define PCIE_DMAREAD_ID
```

Service ID for Pcie_DmaRead() function

Definition at line 139 of file Pcie.h.

6.1.2.8 PCIE_DMAREADINTENABLE_ID

```
#define PCIE_DMAREADINTENABLE_ID
```

Service ID for Pcie_DmaReadIntEnable() function

Definition at line 141 of file Pcie.h.

6.1.2.9 PCIE_DMACHECKREADSTATUS_ID

```
#define PCIE_DMACHECKREADSTATUS_ID
```

Service ID for Pcie_DmaCheckReadStatus() function

Definition at line 143 of file Pcie.h.

6.1.2.10 PCIE_DMAWRITE_ID

```
#define PCIE_DMAWRITE_ID
```

Service ID for Pcie_DmaWrite() function

Definition at line 145 of file Pcie.h.

6.1.2.11 PCIE_DMAWRITEINTENABLE_ID

```
#define PCIE_DMAWRITEINTENABLE_ID
```

Service ID for Pcie_DmaWriteIntEnable() function

Definition at line 147 of file Pcie.h.

6.1.2.12 PCIE_DMACHECKWRITESTATUS_ID

```
#define PCIE_DMACHECKWRITESTATUS_ID
```

Service ID for Pcie_DmaCheckWriteStatus() function

Definition at line 149 of file Pcie.h.

6.1.2.13 PCIE_SENDMSI_ID

```
#define PCIE_SENDMSI_ID
```

Service ID for Pcie_SendMsi() function

Definition at line 151 of file Pcie.h.

6.1.2.14 PCIE_SENDMSIX_ID

```
#define PCIE_SENDMSIX_ID
```

Service ID for Pcie_SendMsiX() function

Definition at line 153 of file Pcie.h.

6.1.2.15 PCIE_DMALLREADSETUP_ID

```
#define PCIE_DMALLREADSETUP_ID
```

Service ID for Pcie_DmaLlReadSetup() function

Definition at line 155 of file Pcie.h.

6.1.2.16 PCIE_DMALLWRITESETUP_ID

```
#define PCIE_DMALLWRITESETUP_ID
```

Service ID for Pcie_DmaLlWriteSetup() function

Definition at line 157 of file Pcie.h.

6.1.2.17 PCIE_GETVERSIONINFO_ID

```
#define PCIE_GETVERSIONINFO_ID
```

Service ID for Pcie_DmaLlWriteSetup() function

Definition at line 160 of file Pcie.h.

6.1.2.18 PCIE_IATU_REGIONS_ALIGN

```
#define PCIE_IATU_REGIONS_ALIGN
```

Alignment requirement for IATU inbound/outbound regions

Definition at line 164 of file Pcie.h.

6.1.3 Function Reference

6.1.3.1 Pcie_Init()

```
void Pcie_Init (
            const Pcie_ConfigType * Config )
```

Initializtion of PCIe driver.

Configures and enables all PCIe instances specified in the configuration structure.

Parameters

in	*Config*	PCIe configuration structure.

6.1.3.2 Pcie_SetOutboundRegion()

```
Std_ReturnType Pcie_SetOutboundRegion (
            uint8 instance,
            const Pcie_OutRegDescriptorType * outboundRegion )
```

Creates an outbound address translation region.

Creates an outbound address translation region using the internal address translation unit of the PCIe module. Accesses in the range defined by srcAddr and srcAddrLim will be translated in a range of the same size starting from dstAddr.

Parameters

in	*instance*	PCIe instance number.
in	*outboundRegion*	pointer to IATU outbound region descripton.

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.3 Pcie_DmaRead()

```
Pcie_StatusType Pcie_DmaRead (
            uint8 instance,
            uint8 channel,
            const Pcie_DmaReadDescriptorType * readDesc )
```

Reads data through DMA.

Initiates an asynchronous read through DMA. Pcie_DmaCheckReadStatus must be called to check if the read is complete.

Parameters

in	*instance*	PCIe instance number. channel DMA channel number. readDesc Transfer descriptor containing all the parameters of the transfer.

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.4 Pcie_DmaReadIntEnable()

```
Std_ReturnType Pcie_DmaReadIntEnable (
            uint8 instance,
            uint8 channel,
            boolean enable )
```

Enables DMA read interrupt.

Enables DMA read interrupt. The interrupt will be triggered when the operation is complete. Pcie_DmaCheck↩ ReadStatus must be called inside the interrupt routine to clear the status flag.

Parameters

in	*instance*	PCIe instance number. channel DMA channel number. enable TRUE to enable interrupts, FALSE to disable.

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.5 Pcie_DmaCheckReadStatus()

```
Pcie_StatusType Pcie_DmaCheckReadStatus (
            uint8 instance,
            uint8 channel,
            uint32 * transfersLeft )
```

Checks the status of a DMA read.

Checks the status of previously started DMA read. Also clears any status flags in case the read is completed.

Parameters

in	*instance*	PCIe instance number. channel DMA channel number.

Returns

Pcie_StatusType

Return values

PCIE_SUCCESS	Read operation was successfully completed.
PCIE_BUSY	Driver busy with a previous operation.
PCIE_ERROR	PCIe device reported an error, operation failed.

6.1.3.6 Pcie_DmaWrite()

```
Pcie_StatusType Pcie_DmaWrite (
            uint8 instance,
            uint8 channel,
            const Pcie_DmaWriteDescriptorType * writeDesc )
```

Writes data through DMA.

Initiates an asynchronous write through DMA. Pcie_DmaCheckWriteStatus must be called to check if the write is complete.

Parameters

in	*instance*	PCIe instance number. channel DMA channel number. writeDesc Transfer descriptor containing all the parameters of the transfer.

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.7 Pcie_DmaWriteIntEnable()

```
Std_ReturnType Pcie_DmaWriteIntEnable (
            uint8 instance,
            uint8 channel,
            boolean enable )
```

Enables DMA write interrupt..

Enables DMA write interrupt. The interrupt will be triggered when the operation is complete. Pcie_DmaCheck↩ WriteStatus must be called inside the interrupt routine to clear the status flag.

Parameters

in	*instance*	PCIe instance number. channel DMA channel number. enable TRUE to enable interrupts, FALSE to disable.

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.8 Pcie_DmaCheckWriteStatus()

```
Pcie_StatusType Pcie_DmaCheckWriteStatus (
            uint8 instance,
            uint8 channel,
            uint32 * transfersLeft )
```

Checks the status of a DMA write.

Checks the status of previously started DMA write. Also clears any status flags in case the write is completed.

Parameters

in	*instance*	PCIe instance number. channel DMA channel number.

Returns

Pcie_StatusType

Return values

PCIE_SUCCESS	Write operation was successfully completed.
PCIE_BUSY	Driver busy with a previous operation.
PCIE_ERROR	PCIe device reported an error, operation failed.

6.1.3.9 Pcie_SendMsi()

```
Std_ReturnType Pcie_SendMsi (
            uint8 instance,
            uint32 intNo )
```

Triggers an MSI interrupt.

Triggers an MSI interrupt if MSI interrupts are enabled for this device.

Parameters

in	*instance*	PCIe instance number. intNo MSI interrupt number.

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.10 Pcie_SendMsiX()

```
Std_ReturnType Pcie_SendMsiX (
            uint8 instance,
            uint32 intNo )
```

Triggers an MSI-X interrupt.

Triggers an MSI-X interrupt if MSI-X interrupts are enabled for this device.

Parameters

in	*instance*	PCIe instance number. intNo MSI-X interrupt number.

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.11 Pcie_DmaLlReadSetup()

```
Std_ReturnType Pcie_DmaLlReadSetup (
            uint8 instance,
            uint8 channel,
            Pcie_DmaLlElementType * llPtr,
            uint32 size )
```

Prepares a DMA read channel for linked list operation.

Prepares a DMA read channel for linked list operation using the memory array provided by the application for the linked list. After this function succesfully completes, Pcie_DmaLlRead can be used to queue read operations. If interrupts are used, Pcie_DmaReadIntEnable must be used to enable interrupts.

Parameters

in	*instance*	PCIe instance number. channel DMA channel number. llPtr Pointer to an array of linked list elements. size Size of the array of linked list elements. Includes Link element, so maximum number of transfers which can be queued is size - 1

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.12 Pcie_DmaLlWriteSetup()

```
Std_ReturnType Pcie_DmaLlWriteSetup (
            uint8 instance,
            uint8 channel,
            Pcie_DmaLlElementType * llPtr,
            uint32 size )
```

Prepares a DMA write channel for linked list operation.

Prepares a DMA write channel for linked list operation using the memory array provided by the application for the linked list. After this function succesfully completes, Pcie_DmaLlWrite can be used to queue write operations. If interrupts are used, Pcie_DmaWriteIntEnable must be used to enable interrupts.

Parameters

in	*instance*	PCIe instance number. channel DMA channel number. llPtr Pointer to an array of linked list elements. size Size of the array of linked list elements. Includes Link element, so maximum number of transfers which can be queued is size - 1

Returns

Std_ReturnType

Return values

E_OK	Operation was successful.
E_NOT_OK	Operation failed.

6.1.3.13 Pcie_GetVersionInfo()

```
void Pcie_GetVersionInfo (
            Std_VersionInfoType * VersionInfo )
```

Returns version information for the driver.

Parameters

in	*VersionInfo*	Driver version information.

6.2 Pcie IPL

6.2.1 Detailed Description

@requirements BSW00374, BSW00379, BSW00318 , DESIGN002

Macros

- #define PCIE_IP_S32G_PCIE_0_BASE_U32

  Pcie 0 base address

- #define PCIE_IP_S32G_PCIE_1_BASE_U32

  Pcie 0 base address

Function Reference

- void Pcie_Ip_Init (uint8 instance, const Pcie_Ip_ConfigType *pcieConfig)

  Initializtion of PCIe driver.

- Pcie_Ip_StatusType Pcie_Ip_SetOutboundRegion (uint8 instance, const Pcie_OutRegDescriptorType *outRegDesc)

  Pcie Set Outbound Region.

- Pcie_Ip_StatusType Pcie_Ip_DmaRead (uint8 instance, uint8 channel, const Pcie_DmaReadDescriptorType *readDesc)

  Pcie DMA read.

- Pcie_Ip_StatusType Pcie_Ip_DmaCheckReadStatus (uint8 instance, uint8 channel, uint32 *transfersLeft)

  Pcie DMA check read status.

- Pcie_Ip_StatusType Pcie_Ip_DmaWrite (uint8 instance, uint8 channel, const Pcie_DmaWriteDescriptor↩ Type *writeDesc)

  Pcie DMA write.

- Pcie_Ip_StatusType Pcie_Ip_DmaCheckWriteStatus (uint8 instance, uint8 channel, uint32 *transfersLeft)

  Pcie DMA check write status.

- Pcie_Ip_StatusType Pcie_Ip_DmaLlReadSetup (uint8 instance, uint8 channel, Pcie_DmaLlElementType *llPtr, uint32 size)

  Prepare read DMA channel for linked list operation.

- Pcie_Ip_StatusType Pcie_Ip_DmaLlWriteSetup (uint8 instance, uint8 channel, Pcie_DmaLlElementType *llPtr, uint32 size)

  Prepare write DMA channel for linked list operation.

- void Pcie_Ip_DmaIRQHandler (uint8 instance)

  PCIe IRQ handler.

6.2.2 Macro Definition Documentation

6.2.2.1 PCIE_IP_S32G_PCIE_0_BASE_U32

```
#define PCIE_IP_S32G_PCIE_0_BASE_U32
```

Pcie 0 base address

Definition at line 107 of file Pcie_Ip.h.

6.2.2.2 PCIE_IP_S32G_PCIE_1_BASE_U32

```
#define PCIE_IP_S32G_PCIE_1_BASE_U32
```

Pcie 0 base address

Definition at line 109 of file Pcie_Ip.h.

6.2.3 Function Reference

6.2.3.1 Pcie_Ip_Init()

```
void Pcie_Ip_Init (
            uint8 instance,
            const Pcie_Ip_ConfigType * pcieConfig )
```

Initializtion of PCIe driver.

Configures and enables all PCIe instances specified in the configuration structure.

Parameters

in	*instance*	PCIe Instance.
in	*pcieConfig*	PCIe configuration structure.

6.2.3.2 Pcie_Ip_SetOutboundRegion()

```
Pcie_Ip_StatusType Pcie_Ip_SetOutboundRegion (
            uint8 instance,
            const Pcie_OutRegDescriptorType * outRegDesc )
```

Pcie Set Outbound Region.

6.2.3.3 Pcie_Ip_DmaRead()

```
Pcie_Ip_StatusType Pcie_Ip_DmaRead (
            uint8 instance,
            uint8 channel,
            const Pcie_DmaReadDescriptorType * readDesc )
```

Pcie DMA read.

6.2.3.4 Pcie_Ip_DmaCheckReadStatus()

```
Pcie_Ip_StatusType Pcie_Ip_DmaCheckReadStatus (
            uint8 instance,
            uint8 channel,
            uint32 * transfersLeft )
```

Pcie DMA check read status.

6.2.3.5 Pcie_Ip_DmaWrite()

```
Pcie_Ip_StatusType Pcie_Ip_DmaWrite (
            uint8 instance,
            uint8 channel,
            const Pcie_DmaWriteDescriptorType * writeDesc )
```

Pcie DMA write.

6.2.3.6 Pcie_Ip_DmaCheckWriteStatus()

```
Pcie_Ip_StatusType Pcie_Ip_DmaCheckWriteStatus (
            uint8 instance,
            uint8 channel,
            uint32 * transfersLeft )
```

Pcie DMA check write status.

6.2.3.7 Pcie_Ip_DmaLlReadSetup()

```
Pcie_Ip_StatusType Pcie_Ip_DmaLlReadSetup (
            uint8 instance,
            uint8 channel,
            Pcie_DmaLlElementType * llPtr,
            uint32 size )
```

Prepare read DMA channel for linked list operation.

6.2.3.8 Pcie_Ip_DmaLlWriteSetup()

```
Pcie_Ip_StatusType Pcie_Ip_DmaLlWriteSetup (
            uint8 instance,
            uint8 channel,
            Pcie_DmaLlElementType * llPtr,
            uint32 size )
```

Prepare write DMA channel for linked list operation.

6.2.3.9 Pcie_Ip_DmaIRQHandler()

```
void Pcie_Ip_DmaIRQHandler (
            uint8 instance )
```

PCIe IRQ handler.

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.